

Coroutine Task Issues

Document #: P3796R1 [[Latest](#)] [[Status](#)]
Date: 2025-08-14
Project: Programming Language C++
Audience: Concurrency Working Group (SG1)
Library Evolution Working Group (LEWG)
Library Working Group (LWG)
Reply-to: Dietmar Kühl (Bloomberg)
<dkuhl@bloomberg.net>

Contents

- [1 Change History](#)
 - [1.1 R0 Initial Revision](#)
 - [1.2 R1: added more issues and discussion](#)
- [2 General](#)
- [3 The Concerns](#)
 - [3.1 affine_on Concerns](#)
 - [3.1.1 affine_on Default Implementation Lacks a Specification](#)
 - [3.1.2 affine_on Semantics Are Not Clear](#)
 - [3.1.3 affine_on's Shape May Not Be Correct](#)
 - [3.1.4 affine_on Shouldn't Forward Stop Requests To Scheduling Operations](#)
 - [3.1.5 affine_on Customisation For Other Senders](#)
 - [3.2 Task Operation](#)
 - [3.2.1 Starting A task Should Not Unconditionally Reschedule](#)
 - [3.2.2 Resuming After A task Should Not Reschedule](#)
 - [3.2.3 No Support For Symmetric Transfer](#)
 - [3.3 Allocation](#)
 - [3.3.1 Unusual Allocator Customisation](#)
 - [3.3.2 Issue: Flexible Allocator Position](#)
 - [3.3.3 Shadowing The Environment Allocator Is Questionable](#)

- 3.4 Stop Token Management
 - 3.4.1 A Stop Source Always Needs To Be Created
 - 3.4.2 The Wording Implies The Stop Token Is Default Constructible
- 3.5 Miscellaneous Concerns
 - 3.5.1 Task Is Not Actually Lazily Started
 - 3.5.2 The Coroutine Frame Is Destroyed Too Late
 - 3.5.3 `task<T, E>` Has No Default Arguments
 - 3.5.4 `bulk` vs. `task_scheduler`
 - 3.5.5 `unhandled_stopped()` Isn't `noexcept`
 - 3.5.6 The Environment Design May Be Inefficient
 - 3.5.7 No Completion Scheduler
 - 3.5.8 Awaitable non-senders Are Not Supported
 - 3.5.9 Should `task::promise_type` Use `with_awaitable_senders`?
 - 3.5.10 A Future Coroutine Feature Could Avoid `co_yield` For Errors
 - 3.5.11 There Is No Hook To Capture/Restore TLS
 - 3.5.12 `return_value` And `return_void` Have No Specification
 - 3.5.13 Consider Supporting `co_return { args... };`
 - 3.5.14 `co_await change_coroutine_scheduler(sched)` Requires Assignable Scheduler
 - 3.5.15 Sender Unaware Coroutines Should Be Able To `co_await` A task
 - 3.5.16 Missing Rvalue Qualification
- 4 Response to “Concerns about the design of `std::execution::task`”
- 5 Conclusion
- 6 Acknowledgement

After the [task proposal](#) was voted to be forwarded to plenary for inclusion into C++26 a number of issues were brought up. The issues were reported using various means. This paper describes the issues and proposes potential changes to address them.

§ 1 Change History

§ 1.1 R0 Initial Revision

§ 1.2 R1: added more issues and discussion

- Added the proposal to support `co_return { ... };` ([link](#))

- Added a response to [P3801r0](#) “Concerns about the design of `std::execution::task`” ([link](#))
- Added a concrete reason why `co_yield` with `with_error(e)` is “clunky” ([link](#))
- Added a discussion of `change_coroutine_scheduler` requiring the scheduler to be assignable ([link](#))
- Added a discussion of adding `operator co_await` ([link](#))
- Added a discussion of bulk vs. `task_scheduler` ([link](#))
- Added discussion of missing rvalue qualification ([link](#))
- Added issue about missing specification of `return_value` and `return_void` ([link](#))
- Various minor fixes

§ 2 General

After LWG voted to forward the [task proposal](#) to be forwarded to plenary for inclusion into C++26 some issues were brought up. The concerns range from accidental omissions (e.g., `unhandled_stopped` lacking `noexcept`) via wording issues and performance concerns to some design questions. In particular the behaviour of the algorithm `affine_on` used to implement scheduler affinity for `task` received some questions which may lead to some design changes and/or clarifications. This document discusses the various issues raised and proposes ways to address some of them.

One statement from the plenary was that `task` is the obvious name for a coroutine task and we should get it right. There are certainly improvements which can be applied. Although I’m not aware of anything concrete beyond the raised issues there is probably potential for improvements. If the primary concern is that there may be better task interfaces in the future and a better task should get the name `task`, it seems reasonable to rename the component. The original name used for the component was `lazy` and feedback from Hagenberg was to rename it `task`. In principle, any name could work, e.g., `affine_task`. It seems reasonable to keep `task` in the name somehow if `task` is to be renamed.

Specifically for `task` it is worth pointing out that multiple coroutine task components can coexist. Due to the design of coroutines and sender/receiver different coroutine tasks can interoperate.

§ 3 The Concerns

Some concerns were posted to the LWG chat on Mattermost (they were originally raised on a Discord chat where multiple of the authors of sender/receiver components coordinate the work). Other concerns were raised separately (and the phrasing isn’t exactly that of the originally, e.g., because it was stated in a different language). I became aware of all but one of these concerns after the poll by LWG to forward the proposal for inclusion into C++26. The rest of this section discusses these concerns:

§ 3.1 `affine_on` Concerns

This section covers different concerns around the specification, or rather lack thereof, of `affine_on`. The algorithm `affine_on` is at the core of the scheduler affinity implementation: this algorithm is used to establish the invariant that a task executes on the currently installed scheduler. Originally, the task proposal used `continue_on` in the specification but during the SG1 discussion it was suggested that a differently named algorithm is used. The original idea was that `affine_on(sndr, sch)` behaves like `continues_on(sndr, sch)` but it is customised to avoid actual scheduling when it knows that `sndr` completes on `sch`. When further exploring the direction of using a different algorithm than `continues_on` some additional potential for changed semantics emerged (see below).

The name `affine_on` was *not* discussed by SG1. The direction was “come up with a name”. The current name just concentrates on the primary objective of implementing scheduler affinity for task. It can certainly use a different name. For example the algorithm could be called `continues_inline_or_on` or `affine_continues_on`.

To some extent `affine_on`’s specification is deliberately vague because currently the execution specification is lacking some tools which would allow omission of scheduling, e.g., for just which completes immediately with a result. While users can’t necessarily tap into the customisations, yet, implementation could use tools like those proposed by P3206 “A sender query for completion behaviour” using a suitable hidden name while the proposal isn’t adopted.

§ 3.1.1 `affine_on` Default Implementation Lacks a Specification

The wording of `affine_on` doesn’t have a specification for the default implementation. For other algorithms the default implementation is specified. To resolve this concern add a new paragraph in `[exec.affine.on]` to provide a specification for the default implementation (probably in front of the current paragraph 5):

- 5 Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If `sender-for<Sndr, affine_on_t>` is false, then the expression `affine_on.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto [_, sch, child] = sndr;
return transform_sender(
    query-with-default(get_domain, sch, default_domain()),
    continues_on(std::move(child), std::move(sch)));
```

except that `sch` is only evaluated once.

The intention for `affine_on` was to all optimisation/customisation in a way reducing the necessary scheduling: if the implementation can determine if a sender completed already on the correct execution agent it should be allowed to avoid scheduling. That may be achieved by using `get_completion_scheduler<set_value_t>` of the sender, using (for now implementation specific) queries like those proposed by P3206 “A sender query for

completion behaviour”, or some other means. Unfortunately, the specification proposed above seems to disallow implementation specific techniques to avoid scheduling. Future revisions of the standard could require some of the techniques to avoid scheduling assuming the necessary infrastructure gets standardised.

§ 3.1.2 **affine_on** Semantics Are Not Clear

The wording in [affine_on p5](#) says:

... Calling `start(op)` will start `sndr` on the current execution agent and execution completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If the current execution resource is the same as the execution resource associated with `sch`, the completion operation on `out_rcvr` may be invoked on the same thread of execution before `start(op)` completes. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

The sentence “If the current execution resource is the same as the execution resource associated with `sch` is not clear to which execution resource is the “current execution resource”. It could be the “current execution agent” that was used to call `start(op)`, or it could be the execution agent that the predecessor completes on.

The intended meaning for “current execution resource” was to refer to the execution resource of the execution agent that was used to call `start(op)`. The specification could be clarified by changing the wording to become:

... Calling `start(op)` will start `sndr` on the current execution agent and execution completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If the ~~current~~ execution resource of the execution agent that was used to invoke `start(op)` is the same as the execution resource associated with `sch`, the completion operation on `out_rcvr` may be called before `start(op)` completes. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

It is also not clear what the actual difference in semantics between `continues_on` and `affine_on` is. The `continues_on` semantics already requires that the resulting sender completes on the specified scheduler's execution agent. It does not specify that it must evaluate a `schedule()` (although that is what the default implementation does), and so in theory it already permits an implementation/customisation to skip the schedule (e.g. if the child sender's completion scheduler was equal to the target scheduler).

The key semantic that we want here is to specify one of two possible semantics that differ from `continues_on`:

1. That the completion of an `affine_on` sender will occur on the same scheduler that the operation started on. This is a slightly stronger requirement than that of `continues_on`, in that it puts a requirement on the caller of `affine_on` to ensure that the

operation is started on the scheduler passed to `affine_on`, but then also grants permission for the operation to complete inline if it completes synchronously.

2. That the completion of an `affine_on` sender will either complete inline on the execution agent that it was started on, or it will complete asynchronously on an execution agent associated with the provided scheduler. This is a slightly more permissive than option 1. in that it permits the caller to start on any context, but also is no longer able to definitively advertise a completion context, since it might now complete on one of two possible contexts (even if in many cases those two contexts might be the same). This weaker semantic can be used in conjunction with knowledge by the caller that they will start the operation on a context associated with the same scheduler passed to `affine_on` to ensure that the operation will complete on the given scheduler.

The description in the paper at the Hagenberg meeting assumed that `task` uses `continues_on` directly to achieve scheduler affinity. During the SG1 discussion it was requested that the approach to scheduler affinity doesn't use `continues_on` directly but rather uses a different algorithm which can be customised separately. This is the algorithm now named `affine_on`. The intention was that `affine_on` can avoid scheduling in more cases than `continues_on`.

It is worth noting that an implementation can't determine the execution resource of the execution agent which invoked `start(op)` directly. If `rcvr` is the receiver used to create `op` it can be queried for `get_scheduler(get_env(rcvr))` which is intended to yield this execution resource. However, there is no guarantee that this is the case [yet?].

The intended direction here is to pursue the second possibility mentioned above. Compared to `continues_on` that would effectively relax the requirement that `affine_on` completes on `sch` if sender doesn't actually change schedulers and completes inline: if `start(op)` gets invoked on an execution agents which matches `sch`'s execution resources the guarantee holds but `affine_on` would be allowed to complete on the execution agent `start(op)` was invoked on. It is upon the user to invoke `start(op)` on the correct execution agent.

Another difference to `continues_on` is that `affine_on` can be separately customised.

§ 3.1.3 `affine_on`'s Shape May Not Be Correct

The `affine_on` algorithm defined in `[exec.affine.on]` takes two arguments; a `sender` and a `scheduler`.

As mentioned above, the semantic that we really want for the purpose in coroutines is that the operation completes on the same execution context that it started on. This way, we can ensure, by induction, that the coroutine which starts on the right context, and resumes on the same context after each suspend-point, will itself complete on the same context.

This then also begs the question: Could we just take the scheduler that the operation will be started on from the `get_scheduler` query on the receivers environment and avoid having to explicitly pass the scheduler as an argument?

To this end, we should consider potentially simplifying the `affine_on` algorithm to just take an input sender and to pick up the scheduler that it will be started on from the receivers environment and promise to complete on that context.

For example, the `await_transform` function could potentially be changed to return: `as_awaitable(affine_on(std::forward<Sndr>(sndr)))`. Then we could define the `affine_on.transform_sender(sndr, env)` expression (which provides the default implementation) to be equivalent to `continues_on(sndr, get_scheduler(env))`.

Such an approach would also require a change to the semantic requirements of `get_scheduler(get_env(rcvr))` to require that `start(op)` is called on that context.

This change isn't strictly necessary, though. The interface of `affine_on` as is can be used. Making this change does improve the design. Nothing outside of `task` currently uses `affine_on` and as it is an algorithm tailored for `task`'s needs it seems reasonable to make it fit this use case exactly. This change would also align with the direction that the execution agent used to invoke `start(op)` matches the execution resource of `get_scheduler(get_env(rcvr))`.

§ 3.1.4 `affine_on` Shouldn't Forward Stop Requests To Scheduling Operations

The `affine_on` algorithm is used by the task coroutine to ensure that the coroutine always resumes back on its associated scheduler by applying the `affine_on` algorithm to each awaited value in a `co_await` expression.

In cases where the awaited operation completes asynchronously, resumption of the coroutine will be scheduled using the coroutines associated scheduler via a `schedule` operation.

If that `schedule` operation completes with `set_value` then the coroutine successfully resumes on its associated execution context. However, if it completes with `set_error` or `set_stopped` then resuming the coroutine on the execution context of the completion is not going to preserve the invariant that the coroutine is always going to resume on its associated context.

For some cases this may not be an issue, but for other cases, resuming on the right execution context may be important for correctness, even during exception unwind or due to cancellation. For example, destructors may require running on a UI thread in order to release UI resources. Or the associated scheduler may be a strand (which runs all tasks scheduled to it sequentially) in order to synchronise access to shared resources used by destructors.

Thus, if a stop-request has been sent to the coroutine, that stop-request should be propagated to child operations so that the child operation it is waiting on can be cancelled if

necessary, but should probably not be propagated to any schedule operation created by the implicit `affine_on` algorithm as this is needed to complete successfully in order to ensure the coroutine resumes on its associated context.

One option to work around this with the status-quo would be to define a scheduler adapter that adapted the underlying `schedule()` operation to prevent passing through stop-requests from the parent environment (e.g. applying the `unstoppable` adapter). If failing to reschedule onto the associated context was a fatal error, you could also apply a `terminate_on_error` adaptor as well.

Then the user could apply this adapter to the scheduler before passing it to the task.

For example:

```
template<std::execution::scheduler S>
struct infallible_scheduler {
    using scheduler_concept = std::execution::scheduler_t;
    S scheduler;
    auto schedule() {
        return
            unstoppable(terminate_on_error(std::execution::schedule(scheduler)));
    }
    bool operator==(const infallible_scheduler&) const noexcept = default;
};

template<std::execution::scheduler S>
infallible_scheduler(S) -> infallible_scheduler<S>;

std::execution::task<void, std::execution::env<>> example() {
    co_await some_cancellable_op();
}

std::execution::task<void, std::execution::env<>> caller() {
    std::execution::scheduler auto sched = co_await
        std::execution::read_env(get_scheduler());
    co_await std::execution::on(infallible_scheduler{sched}, example());
}
```

However, this approach has the downside that this scheduler behaviour now also applies to all other uses of the scheduler - not just the uses required to ensure the coroutines invariant of always resuming on the associated context.

Other ways this could be tackled include:

- Making it the default behaviour of `affine_on` to suppress stop requests for the scheduling operations. That would mean that `affine_on` won't delegate to `continues_on`.
- Somehow making the behaviour a policy decision specified via the Environment template parameter of the task.
- Somehow using domain-based customisation to allow the coroutine to customise the behaviour of `affine_on`.

- Making the `task::promise_type::await_transform` apply this adapter to the scheduler passed to `affine_on`. i.e. it calls `affine_on(std::forward<Sndr>(sndr), infallible_scheduler{SCHED(*sched)})`. Taking this route would mean that the shape of `affine_on` should not be changed.

§ 3.1.5 `affine_on` Customisation For Other Senders

Assuming the the `affine_on` algorithm semantics are changed to just require that it completes either inline or on the context of the receiver environments `get_scheduler` query, then there are probably some other algorithms that could either make use of this, or provide customisations for it that short-circuit the need to schedule unnecessarily.

For example:

- `affine_on(just(args...))` could be simplified to `just(args...)`
- `affine_on(on(sch, sndr))` can be simplified to `on(sch, sndr)` as `on` already provides `affine_on`-like semantics
- The `counting_scope::join` sender currently already provides `affine_on`-like semantics.
 - We could potentially simplify this sender to just complete inline unless the join-sender is wrapped in `affine_on`, in which case the resulting `affine_on(scope.join())` sender would have the semantics that `scope.join()` has today.
 - Alternatively, we could just customise `affine_on(scope.join())` to be equivalent to `scope.join()`.
- Other similar senders like those returned from `bounded_queue::async_push` and `bounded_queue::async_pop` which are defined to return a sender that will resume on the original scheduler.

The intended use of `affine_on` is to avoid scheduling where the algorithm already resumes on a suitable execution agent. However, as the proposal was late it didn't require potential optimisations. The intend was to leave the specification of the default implementation vague enough to let implementations avoid scheduling where they know it isn't needed. Making these a requirement is intended for future revisions of the standard.

It is also a bit unclear how algorithm customisation is actually implemented in practice. Algorithms can advertise a domain via the `get_domain` query which can then be used to transform algorithms: `transform_sender(dom, sender, env...)` [\[exec.snd.transform\]](#) uses `dom.transform_sender(sender, env...)` to transform the sender if this expression is valid (otherwise the transformation from `default_domain` is used which doesn't transform the sender). One way to allow special transformations for `affine_on` is to defined the `get_domain` query for `affine_on` (well, the environment obtained by `get_env(a)` from an `affine_on` sender `a`) to yield a custom domain `affine_on_domain` which delegates transformations of `affine_on(sndr, sch)` the sender `sndr` or the scheduler `sch`:

Let `affine_on_domain.transform_sender(affsndr, env...)` (where `affsndr` is the result of `affine_on(sch, sndr)`) be

- the result of the expression `sndr.affine_on(sch, env...)` if it is valid, otherwise
- the result of the expression `sch.affine_on(sndr, env...)` if it is valid, otherwise
- not defined.

A similar approach would be used for other algorithms which can be customised. Currently, no algorithm defines the exact ways it can be customised in an open form and the intended design for customisations may be different. The above outlines one possible way.

§ 3.2 Task Operation

This section groups three concerns relating to the way task gets started and stopped:

1. task shouldn't reschedule upon `start()`.
2. tasks `co_awaiting` tasks shouldn't reschedule.
3. task doesn't support symmetric Transfer.

§ 3.2.1 Starting A task Should Not Unconditionally Reschedule

In [\[task.promise\] p6](#), the wording for `initial_suspend` says that it unconditionally suspends and reschedules onto the associated scheduler. The intent of this wording seems to be that the coroutine ensures execution initially starts on the associated scheduler by executing a schedule operation and then resuming the coroutine from the initial suspend point inside the `set_value` completion handler of the schedule operation. The effect of this would be that every call to a coroutine would have to round-trip through the scheduler, which, depending on the behaviour of the schedule operation, might relegate it to the back of the schedulers queue, greatly increasing latency of the call.

The specification of `initial_suspend()` (assuming it is rephrased to not resume the coroutine immediately) doesn't really say it unconditionally reschedules. It merely states that an awaiter is returned which arranges for the coroutine to get resumed on the correct scheduler. In general, i.e., when the coroutine gets resumed via `start(os)` on an operation state `os` obtained from `connect(tsk, rcvr)` it will need to reschedule. However, an implementation can pass additional context information, e.g., via implementation specific properties on the receiver's environment: while the `get_scheduler` query may not provide the necessary guarantee that the returned scheduler is the scheduler the operation was started on a different, a non-forwardable query could provide this guarantee.

A possible alternative is to require that `start(op)`, where `op` is the result of `connect(sndr, rcvr)`, is invoked on an execution agent associated with `get_scheduler(get_env(rcvr))`, at least when `sndr` is a `task<...>`. Such a requirement could be desirable more general but it would also be part of the general sender/receiver contract.

The current specification in [\[exec.async.ops\]](#) doesn't have a corresponding constraint. `task<...>` is a sender and where it can be used with standard library algorithms this constraint would hold.

A different way to approach the issue is to use a task-specific awaitable when a task is `co_awaited` by another task. This use of an awaiter shouldn't be observable but it may be better to explicitly spell out that task has a domain with custom transformation of the `affine_on` algorithm and `as_awaitable` operation.

§ 3.2.2 Resuming After A task Should Not Reschedule

Similar to the previous issue, a task needs to resume on the expected scheduler. The definition of `await_transform` which is used for a task is defined in [\[task.promise\] p10](#) (the `co_awaited` task would be the `sndr` parameter):

```
as_awaitable(affine_on(std::forward<Sender>(sndr), SCHED(*this)), *this)
```

As defined there is no additional information about the scheduler on which the `sndr` completes. Thus, it is likely that a scheduling operation is needed after a `co_awaited` task completes when the outer task resumes, even if the `co_awaited` task completed on the same execution agent.

As task is scheduler affine, it is likely that it completes on the same scheduler it was started on, i.e., there shouldn't be a need to reschedule (if the scheduler used by the task was changed using `co_await change_coroutine_scheduler(sch)` it still needs to be rescheduled). The implementation can provide a query on the state used to execute the task which identifies the scheduler on which it completed and the `affine_on` implementation can use this information to decide whether it is necessary to reschedule after the task's completion. It would be preferable if a corresponding query were specified by the standard library to have user-defined senders provide similar information but currently there is no such query.

A different approach is to transform the `affine_on(task, sch)` operation into a task-specific implementation which arranges to always complete on `sch` using task specific information. It may be necessary to explicitly specify or, at least, allow that task provides a domain with a custom transformation for `affine_on`.

§ 3.2.3 No Support For Symmetric Transfer

The specification doesn't mention any use of symmetric transfer. Further, the task gets adapted by `affine_on` in `await_transform` ([\[task.promise\] p10](#)) which produces a different sender than task which needs special treatment to use symmetric transfer. With symmetric transfer stack overflow can be avoided when operation complete immediately, e.g.

```
template <typename Env>
task<void, Env> test() {
    for (std::size_t i{}; i < 1000000; ++i) {
        co_await std::invoke([]()->task<void, env<>> {
```

```

        co_return;
    });
}

```

When using a scheduler which actually schedules the work (rather than immediately completing when a corresponding sender gets started) there isn't a stack overflow but the scheduling may be slow. With symmetric transfer it can be possible to avoid both the expensive scheduling operation and the stack overflow, at least in some cases. When the inner task actually `co_await`s any work which synchronously completes, e.g., `co_await just()`, the code could still result in a stack overflow despite using symmetric transfer. There is a general issue that stack size needs to be bounded when operations complete synchronously. The general idea is to use a trampoline scheduler which bounds stack size and reschedules when the stack size or the recursion depth becomes too big.

Except for the presence or absence of stack overflows it shouldn't be observable whether an implementation invokes the nested coroutine directly through an awaiter interface or using the default implementations of `as_awaitable` and `affine_on`. To address the different scheduling problems (schedule on start, schedule on completion, and symmetric transfer) it may be reasonable to mandate that task customises `affine_on` and that the result of this customisation also customises `as_awaitable`.

§ 3.3 Allocation

§ 3.3.1 Unusual Allocator Customisation

The allocator customisation mechanism is inconsistent with the design of generator allocator customisation: with generator, when you don't specify an allocator in the template arg, then you can use any `allocator_arg` type. With task if no allocator is specified in the Environment the allocator type defaults to `std::allocator<std::byte>` and using an allocator with an incompatible type results in an ill-formed program.

For example:

```

struct default_env {};
struct allocator_env {
    using allocator_type = std::pmr::polymorphic_allocator<>;
};

template <typename Env>
ex::task<int, Env> test(int i, auto&&...) {
    co_return co_await ex::just(i);
}

```

Depending on how the coroutine is invoked this may fail to compile:

- `test<default_env>(17)`: OK - use default allocator
- `test<default_env>(17, std::allocator_arg, std::allocator<int>())`: OK - allocator is convertible

- `test<default_env>(17, std::allocator_arg, std::pmr::polymorphic_allocator<>()):` compile-time error
- `test<allocator_env>(17, std::allocator_arg, std::pmr::polymorphic_allocator<>()):` OK

The main motivator for always having an `allocator_type` is it to support the `get_allocator` query on the receiver's environments when `co_awaiting` another sender. The immediate use of the allocator is the allocation of the coroutine frame and these two needs can be considered separate.

Instead of using the `allocator_type` both for the environment and the coroutine frame, the coroutine frame could be allocated with any allocator specified as an argument (i.e., as the argument following an `std::allocator_arg` argument). The cost of doing so is that the allocator stored with the coroutine frame would need to be type-erased which is, however, fairly cheap as only the `deallocate(ptr, size)` operation needs to be known and certainly no extra allocation is needed to do so. If no `allocator_type` is specified in the `Environment` argument to `task`, the `get_allocator` query would not be available from the environment of received used with `co_await`.

§ 3.3.2 Issue: Flexible Allocator Position

For `task<T, E>` the position of an `std::allocator_arg` can be anywhere in the argument list except it can't be the last argument (as it needs to be followed by an allocator). For generator the `std::allocator_arg` argument needs to be the first argument. That is consistent with other uses of `std::allocator_arg`. `task` should also require the `std::allocator_arg` argument to be the first argument.

The reason why `task` deviates from the approach normally taken is that the consistency with non-coroutines is questionable: the primary reason why the `std::allocator_arg` is needed in the first position is that allocation is delegated to `std::uses_allocator` construction. This approach, however, doesn't apply to coroutines at all: the coroutine frame is allocated from an `operator new()` overloaded for the `promise_type`. In the context of coroutines the question is rather how to make it easy to optionally support allocators.

Any coroutine which wants to support allocators for the allocation of the coroutine frame needs to allow that allocators show up on the parameter list. As coroutines normally have other parameters, too, requiring that the optional `std::allocator_arg/allocator` pair of arguments comes first effectively means that a forwarding function is provided, e.g., for a coroutine taking an `int` parameter for its work (Env is assumed to be a environment type with a suitable definition of `allocator_type`):

```
task<void, Env> work(std::allocator_arg_t, auto, int value) {
    co_await just(value);
}
task<void, Env> work(int value) {
    return work(std::allocator_arg, std::allocator<std::byte>());
}
```

If the `allocator_arg` argument can be at an arbitrary location of the parameter list, defining a coroutine with optional allocator support amounts to adding a suitable trailing list of parameters, e.g.:

```
task<void, Env> work(int value, auto&&...) {  
    co_await just(value);  
}
```

Constraining `task` to match the use of `generator` is entirely possible. It seems more reasonable to rather consider relaxing the requirement on `generator` in a future revision of the C++ standard.

§ 3.3.3 Shadowing The Environment Allocator Is Questionable

The `get_allocator` query on the environment of the receiver passed to `co_awaited` senders always returns the allocator determined when the coroutine frame is created. The allocator provided by the environment of the receiver the task is connected to is hidden.

Creating a coroutine (calling the coroutine function) chooses the allocator for the coroutine frame, either explicitly specified or implicitly determined. Either way the coroutine frame is allocated when the function is called. At this point the coroutine isn't connected to a receiver, yet. The fundamental idea of the allocator model is that children of an entity use the same allocator as their parent unless an allocator is explicitly specified for a child. The `co_awaited` entities are children of the coroutine and should share the coroutine's allocator.

In addition, to forward the allocator from the receiver's environment in an environment, its static type needs to be convertible to the coroutine's allocator type: the coroutine's environment types are determined when the coroutine is created at which point the receiver isn't known, yet. Whether the allocator from the receiver's environment can be used with the statically determined allocator type can't be determined. It may be possible to use a type-erase allocator which could be created in the operation state when the task is connected.

On the flip side, the receiver's environment can contain the configuration from the user of a work-graph which is likely better informed about the best allocator to use. The allocator from the receiver's environment could be forwarded if the task's allocator can be initialised with it, e.g., because the task use `std::pmr::polymorphic_allocator<>`. It isn't clear what should happen if the receiver's environment has an allocator which can't be converted to the allocator based task's environment: at least ignoring a mismatching allocator or producing a compile time error are options. It is likely possible to come up with a way to configure the desired behaviour using the environment.

§ 3.4 Stop Token Management

§ 3.4.1 A Stop Source Always Needs To Be Created

The specification of the `promise_type` contains exposition-only members for a stop source and a stop token. It is expected that in many situations the upstream stop token will be an `inplace_stop_token` and this is also the token type exposed downstream. In these cases the `promise_type` should store neither a stop source nor a stop token: instead the stop token should be obtained from the upstream environment. Necessarily storing a stop source and a stop token would increase the size of the promise type by multiple pointers. On top of that, to forward the state of an upstream stop token via a new stop source requires registration/deregistration of a stop callback which requires dealing with synchronisation.

The expected implementation is, indeed, to get the stop token from the operation state: when the operation state is created, it is known whether the upstream stop token is compatible with the statically determined stop token exposed by task to `co_awaited` operations via the respective receiver's environment. If the type is compatible there is no need to store anything beyond the receiver from which a stop token can be obtained using the `get_stop_token` query when needed. Otherwise, the operation state can store an optional stop source which gets initialised and connected to the upstream stop token via a stop callback when the first stop token is requested.

The exposition-only members are not meant to imply that a corresponding object is actually stored or where they are. Instead, they are merely meant to talk about the corresponding entities where the behaviour of the environment is described. If the current wording is considered to imply that these entities actually exist or it can be misunderstood to imply that, the wording may need some massaging possibly using specification macros to refer to the respective entities in the operation state.

§ 3.4.2 The Wording Implies The Stop Token Is Default Constructible

Using an exposition-only member for the stop token implies that the stop token type is default constructible. The stop token types are generally not default constructible and are, instead, created via the stop source and always refer to the corresponding stop source.

The intent here is, indeed, that the stop token type isn't actually stored at all: the operation state either stores a stop source which is used to get the stop token or, probably in the comment case, the stop token is obtained from the upstream receiver's environment by querying `get_stop_token`. The rewording for the previous concern should also address this concern.

§ 3.5 Miscellaneous Concerns

The remaining concerns aren't as coupled to other concerns and discussed separately.

§ 3.5.1 Task Is Not Actually Lazily Started

The wording for `task<...>::promise_type::initial_suspend` in `[task.promise]` p6 may imply that a task is eagerly started:


```
auto initial_suspend() noexcept;
```

Returns: An awaitable object of unspecified type ([`expr.await`]) whose member functions arrange for

- the calling coroutine to be suspended,
- the coroutine to be resumed on an execution agent of the execution resource associated with `SCHED(*this)`.

In particular the second bullet can be interpreted to mean that the task gets resumed immediately. That wouldn't actually work because `SCHED(*this)` only gets initialised when the task gets connected to a suitable receiver. The intention of the current specification is to establish the invariant that the coroutine is running on the correct scheduler when the coroutine is resumed (see discussion of `affine_on` below). The mechanisms used to achieve that are not detailed to avoid requiring that it gets scheduled. The formulation should, at least, be improved to clarify that the coroutine isn't resumed immediately, possibly changing the text like this:

- the coroutine ~~to be resumed~~resuming on an execution agent of the execution resource associated with `SCHED(*this)` when it gets resumed.

The proposed fix for the issue is to specify that `initial_suspend()` always returns `suspend_always{}` and require that `start(...)` calls `handle.resume()` to resume the coroutine on the appropriate scheduler after `SCHED(*this)` has been initialised. The corresponding change could be

Change [\[task.promise\] p6](#):

```
auto initial_suspend() noexcept;
```

Returns: An awaitable object of unspecified type ([`expr.await`]) whose member functions arrange for:

- the calling coroutine to be suspended,
- the coroutine to be resumed on an execution agent of the execution resource associated with `SCHED(*this)`.

Returns: `suspend_always{}`.

The suggestion is to ensure that the task gets resumed on the correct associated context via added requirements on the receiver's `get_scheduler()` (see below).

In separate discussions it was suggested to relax the specification of `initial_suspend()` to allow returning an awaiter which does semantically what `suspend_always` does but isn't necessary `suspend_always`. The proposal was to copy the wording of the `suspend_always` specification [\[coroutine.trivial.awaitables\]](#). This specification just shows the class completion definition.

§ 3.5.2 The Coroutine Frame Is Destroyed Too Late

When the task completes from a suspended coroutine without ever reaching `final_suspend` the coroutine frame lingers until the operation state object is destroyed. This happens when the task isn't resumed because a `co_awaited` sender completed with `set_stopped()` or when the task completes using `co_yield` `when_error(e)`. In these cases the order of destruction of objects may be unexpected: objects held by the coroutine frame may be destroyed only long after the respective completion function was called.

This behaviour is an oversight in the specification and not intentional at all. Instead, there should be a statement that the coroutine frame is destroyed before any of the completion functions is invoked. The implication is that the results can't be stored in the promise type but that isn't a huge constraint as the best place store them is in the operation state.

§ 3.5.3 `task<T, E>` Has No Default Arguments

The current specification of `task<T, E>` doesn't have any default arguments in its first declaration in [\[execution.syn\]](#). The intent was to default the type `T` to `void` and the environment `E` to `env<>`. That is, this change should be applied to [\[execution.syn\]](#):

```
...
// [exec.task]
template <class T=void, class Environment=env<>>
class task;
...
```

It isn't catastrophic if that change isn't made but it seems to improve usability without any drawbacks.

§ 3.5.4 `bulk` vs. `task_scheduler`

Normally, the scheduler type used by an operation can be deduced when a sender is connected to a receiver from the receiver's environment. The body of a coroutine cannot know about the receiver the task sender gets connected to. The implication is that the type of the scheduler used by the coroutine needs to be known when the task is created. To still allow custom schedulers used when connecting, the type-erase scheduler `task_scheduler` is used. However, that leads to surprises when algorithms are customised for a scheduler as is, e.g., the case for `bulk` when used with a `parallel_scheduler`: if `bulk` is `co_awaited` within a coroutine using `task_scheduler` it will use the default implementation of `bulk` which sequentially executes the work, even if the `task_scheduler` was initialised with a `parallel_scheduler` (the exact invocation may actually be slightly different or need to use `bulk_chunked` or `bulk_unchunked` but that isn't the point being made):

```
struct env {
    auto query(ex::get_scheduler_t) const noexcept { return ex::parallel_scheduler(); }
};
struct work {
```

```

    auto operator()(std::size_t s){ /*...*/ };
};

ex::sync_wait(
    ex::write_env(ex::bulk(ex::just(), 16u, work{}),
        env{}
    ));
ex::sync_wait(ex::write_env(
    []()->ex::task<void, ex::env<>> { co_await ex::bulk(ex::just(), 16u,
        work{}); }(),
    env{}
));

```

The two invocations should probably both execute the work in parallel but the coroutine version doesn't: it uses the `task_scheduler` which doesn't have a specialised version of `bulk` to potentially delegate in a type-erased form to the underlying scheduler. It is straight forward to move the `write_env` wrapper inside the coroutine which fixes the problem in this case but this need introduces the potential for a subtle performance bug. The problem is sadly not limited to a particular scheduler or a particular algorithm: any scheduler/algorithm combination which may get specialised can suffer from the specialised algorithm not being picked up.

There are a few ways this problem can be addressed (this list of options is almost certainly incomplete):

1. Accept the situation as is and advise users to be careful about customised algorithms like `bulk` when using `task_scheduler`.
2. Extend the interface of `task_scheduler` to deal with a set of algorithms for which it provides a type-erased interface. The interface would likely be more constrained and it would use virtual dispatch at run-time. However, the set of covered algorithms would necessarily be limited in some form.
3. To avoid the trap, make the use of known algorithms incompatible with the use of `task_scheduler`, i.e., "customise" these algorithms for `task_scheduler` such that a compile-time error is produced.

A user who knows that the main purpose of a coroutine is to executed an algorithm customised for a certain scheduler can use `task<T, E>` with an environment `E` specifying exactly that scheduler type. However, this use may be nested within some sender being `co_awaited` and users need to be aware that the customisation wouldn't be picked up. Any approach I'm currently aware of will have the problem that customised versions of an algorithm are not used for algorithms we are currently unaware of.

§ 3.5.5 `unhandled_stopped()` Isn't `noexcept`

The `unhandled_stopped()` member function of `task::promise_type` [[task.promise](#)] is not currently marked as `noexcept`.

As this method is generally called from the `set_stopped` completion-handler of a receiver (such as in [\[exec.as.awaitable\] p4.3](#)) and is invoked without handler from a `noexcept` function, we should probably require that this function is marked `noexcept` as well.

The equivalent method defined as part of the `with_awaitable_senders` base-class ([\[exec.with.awaitable.senders\] p1](#)) is also marked `noexcept`.

This change should be applied to [\[task.promise\]](#):

```
...
void uncaught_exception();
coroutine_handle<> unhandled_stopped() noexcept;

void return_void(); // present only if is_void_v<T> is true;
...
```

...

```
coroutine_handle<> unhandled_stopped() noexcept;
```

- 13 *Effects*: Completes the asynchronous operation associated with `STATE(*this)` by invoking `set_stopped(std::move(RCVR(*this)))`.

§ 3.5.6 The Environment Design May Be Inefficient

The environment returned from `get_env(rcvr)` for the receiver used to connect to `co_awaited` sender is described in terms of members of the `promise_type::state` in [\[task.promise\] p15](#). In particular the `state` contains a member `own-env` which gets initialised via the upstream receiver's environment (if the corresponding expression is valid). As the environment `own-env` is initialised with a temporary, `own-env` will need to create a copy of whatever it is holding.

As the environment exposed to `co_awaited` senders via the `get_env(rcvr)` is statically determined when the task is created, not when the task is connected to an upstream receiver, the environment needs to be type erase. Some of the queries (`get_scheduler`, `get_stop_token`, and `get_allocator`) are known and already receive treatment by the task itself. However, the task cannot know about user-defined properties which may need to be type-erased as well. To allow the Environment to possibly use type-erase properties from the upstream receiver, an `own-env-t` object is created and a reference to the object is passed to the Environment constructor (if there are suitable constructors). Thus, the entire use of `own-env-t` is about enabling storage of whatever is needed to type-erase the result of user-defined queries. Ideally, this functionality isn't needed and the Environment parameter doesn't specify an `env_type`. If it is needed the type-erasure will likely need to store some data.

§ 3.5.7 No Completion Scheduler

The concern raised is that task doesn't define a `get_env` that returns an environment with a `get_completion_scheduler<Tag>` query. As a result, it will be necessary to reschedule

in some situations although the task already completes on the correct scheduler.

The query `get_completion_scheduler(env)` operates on the sender's environment, i.e., it would operate on the task's environment. However, when the task is created it has no information about the scheduler, yet, it is going to use. The entire information the task has is what is passed to the coroutine [factory] function. The information about any scheduling gets provided when the task is connected to a receiver: the receiver provides the queries on where the task is started (`get_scheduler`). The task may complete on this scheduler assuming the scheduler isn't changed (using `co_await change_coroutine_scheduler(sch)`).

The only place where a task actually knows its completion scheduler is once it has completed. However, there is no query or environment defined for completed operation states. Thus, it seems there is no way where a `get_completion_scheduler` would be useful. A future evolution of the sender/receiver interface may change that in which case task should be revisited.

§ 3.5.8 Awaitable non-senders Are Not Supported

The overload of `await_transform` described in [task.promise] is constrained to require arguments to satisfy the `sender` concept. However, this precludes awaiting types that implement the `as_awaitable()` customisation point but that do not satisfy the `sender` concept from being able to be awaited within a task coroutine.

This is inconsistent with the behaviour of the `with_awaitable_senders` base class defined in [exec.with.awaitable.senders], which only requires that the awaited value supports the `as_awaitable()` operation.

The rationale for this is that the argument needs to be passed to the `affine_on` algorithm which currently requires its argument to model `sender`. This requirement in turn is there because it is unclear how to guarantee scheduler affinity with an awaitable-only interface without wrapping a coroutine around the awaitable.

Do we want to consider relaxing this constraint to be consistent with the constraints on [exec.with.awaitable.senders]?

This would require either:

- relaxing the `sender` constraint on the `affine_on` algorithm to also allow an argument that has only an `as_awaitable()` but that did not satisfy the `sender` concept.
- extending the `sender` concept to match types that provide the `.as_awaitable()` member-function similar to how it supports types with `operator co_await()` (see [exec.connect]).

§ 3.5.9 Should `task::promise_type` Use `with_awaitable_senders`?

The existing `[exec]` wording added the `with_awaitable_senders` helper class with the intention that it be usable as the base-class for asynchronous coroutine promise-types to provide the ability to await senders. It does this by providing the necessary `await_transform` overload and also the `unhandled_stopped` member-function necessary to support the `as_awaitable()` adaptor for senders.

However, the current specification of `task::promise_types` does not use this facility for a couple of reasons:

- It needs to apply the `affine_on` adaptor to awaited senders.
- It needs to provide a custom implementation of `unhandled_stopped` that reschedules onto the original scheduler in the case that it is different from the current scheduler (e.g. due to `co_await change_coroutine_scheduler{other}`).

This raises a couple of questions:

- Should there also be a `with_affine_awaitable_senders` that implements the scheduler affinity logic?
- Is `with_awaitable_senders` actually the right design if the first coroutine type we add to the standard library doesn't end up using it?

These are valid questions. It seems the `with_awaitable_senders` class was designed at time when there was no consensus that a `task` coroutine should be scheduler affine by default. However, these questions don't really affect the design of `task` and they can be answered in a future revision of the standard. The `task` specification also uses a few other tools which could be useful for users who want to implement their own custom `task`-like coroutine. Such tools should probably be proposed for inclusion into a future revision of the C++ standard, too.

However, what is relevant to the `task` specification is that the wording for `unhandled_stopped` in `[task.promise]` paragraph 13 doesn't spell out that the `set_stopped` completion is invoked on the correct scheduler.

§ 3.5.10 A Future Coroutine Feature Could Avoid `co_yield` For Errors

The mechanism to complete with an error without using an exception uses `co_yield with_error(x)`. This use may surprise users as `co_yielding` is normally assumed not to be a final operation. A future language change may provide a better way to achieve the objective of reporting errors without using exceptions.

Using not, yet, proposed features which may become part of a future revision of the C++ standard may always provide facilities which result in a better design for pretty much anything. There is no current `task` implementation and certainly none which promises ABI stability. Assuming a corresponding language change gets proposed reasonably early for the next cycle, implementations can take it into account for possibly improving the interface without the need to break ABI. The shape of the envisioned language change is unknown but most likely it is an extension which hopefully integrates with the existing de-

sign. The functionality to use `co_yield` would, however, continue to exist until it eventually gets deprecated and removed (assuming a better alternative emerges).

§ 3.5.11 There Is No Hook To Capture/Restore TLS

This concern is the only one I was aware of for a few weeks prior to the Sofia meeting. I believe the necessary functionality can be implemented although it is possibly harder to do than with more direct support. The concern raised is that existing code accesses thread local storage (TLS) to store context information. When a `co_await` actually suspends it is possible that a different task running on the same thread replaces used TLS data or the task gets resumed on a different thread of a thread pool. In that case it is necessary to capture the TLS variables prior to suspending and restoring them prior to resuming the coroutine. There is currently no special way to actually do that.

The sender/receiver approach to propagating context information isn't using TLS but rather the environments accessed via the receiver. However, existing software does use TLS and at least for a migration period corresponding support may be necessary. One way to implement this functionality is using a scheduler adapter with a customised `affine_on` algorithm:

- When the `affine_on` algorithm is started, it captures the relevant TLS data into the operation state and then starts `affine_on` for the adapted scheduler with a receiver observing the completions.
- When the adapted scheduler invokes any of the completion signals the TLS data is restored before invoke the algorithm's completion signal.

Support for optionally capturing some state before starting a child operation and restoring the captured state before resuming could be added to the task, avoiding the need to use custom scheduling. Doing so would yield a nicer and easier to use interface. In environments where TLS is currently used and developers move to an asynchronous model, failure to capture and restore data in TLS is a likely source of errors. However, it will be necessary to specify what data needs to be stored, i.e., the problems can't be automatically avoided.

§ 3.5.12 `return_value` And `return_void` Have No Specification

The functions `return_value` and `return_void` declared for the `task<...>::promise_`-type in `[task.promise]` (only one of them is present in a given `promise_type`) are entirely missing from the specification. To fix this problem the specifications need to be added:

```
void return_void();           // present only if is_void_v<T> is
                             true;
```

x *Effects:* does nothing.

```
template<class V>
void return_value(V&& value); // present only if is_void_v<T> is
                             false;
```

x+1 *Effects: Equivalent to `result.emplace(std::forward(value));`*

§ 3.5.13 Consider Supporting `co_return { args... };`

The current [declaration of `return\_value`](#) in the `promise_type` specifies the function without a default type for the template parameter:

```
template <class V>
void return_value(V&& value); // present only if is_void<T> is false
```

As a result it isn't possible to use aggregate initialisation without mentioning the type. For example, the following code isn't valid:

```
struct aggregate { int value{0} };
[]() -> task<aggregate, std::env<>> { co_return {}; };
[]() -> task<aggregate, std::env<>> { co_return { 42 }; };
```

To better match the normal functions, the template parameter should be defaulted which would make the code above valid:

```
template <class V = T>
void return_value(V&& value); // present only if is_void<T> is false
```

There isn't anything broken with the specification but adding the default type improves usability. If necessary, this change can be applied in a future revision of the standard. It would be nice to fix it for C++26.

§ 3.5.14 `co_await` `change_coroutine_scheduler(sched)` Requires Assignable Scheduler

The specification of `change_coroutine_scheduler(sched)` uses `std::exchange` to put the scheduler into place (in [\[task.promise\] p11](#)):

```
auto    await_transform(change_coroutine_scheduler<Sch>    sch)
noexcept;
```

11 Effects: Equivalent to:

```
return await_transform(just(exchange(SCHED(*this), scheduler_
type(sch.scheduler))), *this);
```

The problem is that `std::exchange(x, v)` expects `x` to be assignable from `v` but there is no requirement for scheduler to be assignable. It would be possible to specify the operation in a way avoiding the assignment:

```
return await_transform(just(exchange(SCHED(*this), scheduler_
type(sch.scheduler))), *this);
auto* s{address_of(SCHED(*this))};
auto rc{std::move(*s)};
s->~scheduler_type();
new(s) scheduler_type(std::move(sch));
return std::move(rc);
```


An alternative is to consider the presence of the assignment to be an indicator of whether the scheduler can be replaced: the possibility of changing the scheduler used for scheduler affinity means that the body of the coroutine may actually complete on a different scheduler than the scheduler it was originally started on. It can be determined statically whether `co_await change_coroutine_scheduler(s)` was used. As a result, when a task awaits another task it is potentially necessary to scheduler the continuation for the outer task. This possibility and the corresponding check may have a cost. However, when the scheduler isn't assignable, it couldn't be replaced and that feature is statically checkable, i.e., any cost associated with the potential of the inner task changing the scheduler is avoided.

It should be possible to relax the requirements for replacing schedulers in a future revision of the standard, i.e., this issues doesn't necessarily need to be resolved one way or the other for C++26.

§ 3.5.15 Sender Unaware Coroutines Should Be Able To `co_await` A task

The request here is to add an `operator co_await()` to `task` which returns an awaiter used to start the task. On the surface doing so should allow any coroutine to `co_await` a task. Unfortunately, here are a few complications:

1. If the `task<T, E>` is used with an environment `E` whose `scheduler_type` can't be default constructed the environment associated with the promise type `P` from the `std::coroutine_handle<P>` passed to `await_suspend` needs to have a `get_env` providing an environment with a query for `get_scheduler`. The default used (`task_scheduler`) is not default constructible because by default `task<T, E>` should be scheduler affine and to implement that the scheduler needs to be known. The implication is that the `operator co_await` can only be used when either the `task<T, E>` isn't scheduler affine or the `co_awaiting` coroutine knows at least about the `get_scheduler` query, i.e., it isn't entirely sender agnostic.
2. When `co_awaiting` a sender within a `task<T, E>` it is always possible that a sender completes with `set_stopped()`, i.e., the coroutine gets cancelled. However, to actually implement that, the `co_awaiting` coroutine needs to have a hook which can be invoked to notify it about this cancellation. When using `as_awaitable(s)` this hook is to call `unhandled_stopped()` on the promise object. Thus, a sender agnostic coroutine trying to `co_await` a `std::task<T, E>` would need to know about this protocol or only support `task<T, E>` objects for which the `co_await`s cannot result in completing with `set_stopped()`.
3. Other senders are not `co_awaitable` by sender agnostic coroutines. If a task wants to support `co_awaiting` senders it needs to provide an `await_transform` which turns a sender into an awaitable, e.g., using `as_awaitable`. Why should that be different for `task`? As `task` is already a coroutine it seems this is a fairly weak argument, though.

Given these constraints it is unclear whether it is worth adding an `operator co_await()` to `task`. If a user really wants to `co_await` a `task<T, E>`, it should be possible by adapt-

ing the object to provide a `get_scheduler` query (e.g., using `std::write_env`), to deal with the potential of a `set_stopped()` completion (e.g., using `upon_stopped`), and then doing something similar to `as_awaitable` with the result, just without the need to have an `unhandled_stopped`. As there are various choices of what schedulers should be provided and how to deal with the `set_stopped()` completion it seems reasonable to not add support for `operator co_await()` at this time.

§ 3.5.16 Missing Rvalue Qualification

The nested sender of a `task_scheduler` isn't necessary copyable. As the operation is type erased, the `task_scheduler::ts-sender::connect` should be rvalue qualified, i.e., in [\[exec.task.scheduler\] p8](#) add rvalue qualification to `connect`:

```
namespace execution {
    class task_scheduler::ts-sender { // exposition only
    public:
        using sender_concept = sender_t;

        template <receiver R>
            state<R> connect(R&& rcvr)&&;
    };
}
```

In [\[exec.task.scheduler\] p10](#) add rvalue qualification to `connect`:

```
template<receiver Rcvr>
    state<Rcvr> connect(Rcvr&& rcvr)&&;
```

Coroutines aren't copyable. When using objects holding a `coroutine_handle<P>` such that the underlying `coroutine_handle<P>` gets transferred to a different object, the operation should be rvalue qualified and not all such operations currently are rvalue qualified. The `task<T, E>::connect` function should be rvalue qualified, i.e., change [\[task.class\] p1](#):

```
namespace std::execution {
    template <class T, class Environment>
    class task {
    ...
        template <receiver R>
            state<R> connect(R&& rcv)&&;
    ...
    };
}
```

Change [\[task.members\] p3](#):

```
template <receiver R>
    state<R> connect(R&& rcv)&&;
```

If an `as_awaitable` member is added to `task` (see [this issue](#)) this member should also be rvalue qualified.

§ 4 Response to “Concerns about the design of `std::execution::task`”

The paper “Concerns about the design of `std::execution::task`” raises three major points and three minor points. The three major points are:

1. Synchronous completions can result in stack overflow. This problem was discussed in the proposal “Add a Coroutine Task Type”. If a coroutine uses an actually scheduling scheduler there is no issue with stack overflow unless an implementation “optimises” `affine_on` to complete synchronously in some cases and they don’t guard against stack overflow, e.g., using a “trampoline scheduler”. Also, the danger of stack overflow isn’t actually specific to `task` but applies to any synchronous completion, especially if it happens from a loop-like construct. In that sense the issue exists without `task` although using a `task` may make it more likely that users encounter this issue.

The concern also mentioned that there is no support for symmetric transfer between tasks. This point is already discussed above in section [No Support For Symmetric Transfer](#).

2. One concern addresses that the life-time of variables local to coroutines is surprising: there is no requirement to destroy the coroutine frame before completing the operation. This issue is discussed in [The Coroutine Frame Is Destroyed Too Late](#) and the fix is to explicitly specify that the coroutine frame has to be destroyed before completing the operation. The only implication is that the result of the operation needs to be stored in the operation state rather than the promise type which is, however, a reasonable approach anyway.
3. It is raised as a concern that that `task` doesn’t defined against capturing reference which may be out of scope before the task is executed. Here is a minimal example demonstrating the problem (the example in the paper uses `co` instead of `t` and omitted the move but it is clear what is intended):

```
task<void, env<>> g() {
    auto t = [] (const int& v) -> task<int, env<>> { co_return v; }
        (42);
    auto v = co_await std::move(t);
}
```

The subtle problem with this code is the argument `42` is turned into a temporary `int` which is captured by `const int&` in the coroutine frame. Once the expression completes this temporary goes out of scope and is destroyed. When `t` gets `co_awaited` the body of the function accesses an already destroyed object. It is worth noting that the relevant references can be hidden, i.e., they don’t necessarily show up in the signature of the coroutine (preventing `const&` parameters isn’t a solution).

This problem and an approach to avoid it is explained at length in Aaron Jacobs's C++Now 2024 talk “[Coroutines at Scale](#)”. The proposed solution is to return an object which can only be `co_awaited` immediately. While having such a coroutines would certainly be beneficial and it could be the recommended coroutine for use from within another coroutine, it isn't a viable replacement for `task` in all contexts:

1. It is intended that `tasks` can be used as arguments to sender algorithms, e.g., to `when_all` to execute multiple asynchronous operations potentially concurrently, e.g.:

```
task<void, env<>> do_work(std::string value) { /* work */
    co_return; }
task<void, env<>> execute_all() {
    co_await when_all(
        do_work("arguments 1"),
        do_work("arguments 2")
    );
};
```

2. It is intended that `tasks` can be used with `counting_scope` (although to do so it is necessary to provide an environment with a scheduler). There is no scoping relationship between the `counting_scope` and the argument to `spawn`.
3. It is intended that `task<T, E>` is used to encapsulate the definition of an asynchronous operation into a function which can be implemented in a separate translation unit and that corresponding objects are returned from functions.

While the idea of limiting the scope of `task` was considered (admittedly that isn't reflected in the proposal paper), I don't think there is a way to incorporate the proposed safety mechanism into `task`. It can be incorporated into a different coroutine type which can be `co_awaited` by `task`, though.

The three minor points are:

1. The use of `co_yield` with `with_error(e)` is “clunky”. There is no disagreement here. A concrete reason why `co_yield` with `with_error(e)` isn't ideal is that `co_yield` cannot be used within a `catch` block. In a future revision of the C++ standard it may become possible that `co_return` with `with_error(e)` can be supported (currently that isn't possible in general because a promise type cannot provide both `return_value` and `return_void` functions). However, using `co_return` for both errors and normal returns introduces a small problem when returning an object of type `with_error` as normal a result (this is, however, a rather weird case):

```
struct error_env {
    using error_types = completion_signatures<set_error_t(int)>;
};
[]() -> task<with_error<int>, error_env> {
    co_return with_error(17); // error or success?
}
```

2. The use of `co_await scheduler(sched);` does not result in the coroutine being resumed on `sched`: due to scheduler affinity the coroutine is resumed on the coroutine's scheduler, not on `sched` (unless, of course, these two scheduler are identical). To change a scheduler something else, specifically `co_await change_coroutine_scheduler(sched)`, needs to be used. The use of `co_await scheduler(sched);` was used by a sender/receiver library to replace the coroutine's scheduler and the recommendation from this experience was to not use this exact approach. The hinted at proposal is to use `co_yield sched;` to change the scheduler: just because `co_yield` is used for returning errors using `with_error(e)` doesn't mean it can't be used for other uses and `co_yield x;` where `x` is a scheduler could change the scheduler. However, it is quite intentional to use an explicitly visible type like `change_coroutine_scheduler` to make changing the scheduler visible. I have no objection to using `co_yield change_coroutine_scheduler(sched);` instead of using `co_await` if that is the preference.
3. Coroutine cancellation is ad-hoc: the `std::execution` proposal introduced the use of `unhandled_stopped` to deal with cancellation. This use wasn't part of the [task proposal](#). I don't think it is reasonable to not have a coroutine task because there could be a nicer language level approach in the future.

I think I understand the concerns raised. I don't think that any of them warrants removing the task from the standard document. The argument that we should only standardise perfect design would actually lead to hardly anything getting standardised because pretty much everything is imperfect on some level.

§ 5 Conclusion

There are some parts of the specification which can be misunderstood. These should be clarified correspondingly. In most of these cases the change only affects the wording and doesn't affect the design.

For the issues around `affine_on` there are some potential design changes. In particular with respect to the exact semantics of `affine_on` the design was possibly unclear. Likewise, explicitly specifying that task customises `affine_on` and provides an `as_awaitable` function is somewhat in design space. The intention was that doing so would be possible with the current specification and it just didn't spell out how `co_await` a task from a task actually works.

However, some fixes of the specification are needed.

§ 6 Acknowledgement

The issue descriptions are largely based on [this draft](#) written by Lewis Baker. Lewis Baker and Tomasz Kamiński contributed to the discussions towards addressing the issues.